

CS-4063: Natural Language Processing

Assignment 3 — Transformers + RAG

Retrieval-Augmented Generation for Amazon Product Review Analysis

Student ID: f230647

Course: CS-4063 Natural Language Processing

Institution: FAST-NUCES

Date: April 29, 2026

Repository: github.com/zaidmuslim0647/f230647-NLP-Assignment3

Abstract

This report describes the design and implementation of a three-stage Retrieval-Augmented Generation (RAG) pipeline for Amazon product review analysis. All transformer components are implemented from scratch using PyTorch without relying on `nn.Transformer`, `nn.MultiheadAttention`, or any pretrained model. The pipeline consists of: (A) an encoder-only transformer trained with multi-task learning (sentiment classification and helpfulness prediction), (B) a cosine-similarity retrieval module operating over the encoder's training-set embeddings, and (C) a decoder-only transformer that autoregressively generates a natural-language explanation conditioned on the input review, its predicted labels, and top- k retrieved similar reviews. The system is trained and evaluated on a 36 000-sample subset of the Amazon Reviews dataset spanning three product categories.

Contents

1	System Overview and Architecture	2
1.1	Shared Transformer Components	2
2	Dataset and Preprocessing	3
2.1	Dataset Construction	3
2.2	Preprocessing Pipeline	3
3	Part A: Encoder-Only Transformer	3
3.1	Architecture	3
3.2	Multi-Task Learning	4
3.3	Training Configuration	4
3.4	Results	4
4	Part B: Retrieval Module	5
4.1	Design	5
4.2	Retrieval Quality Analysis	5
5	Part C: Decoder-Only Transformer	6
5.1	Architecture	6
5.2	Input Template	6
5.3	Training Configuration	7
5.4	Evaluation	7
6	Design Decision Justifications	8
7	Hyperparameter Tuning	8
8	Limitations and Future Work	9

1 System Overview and Architecture

Figure 1 illustrates the end-to-end pipeline.

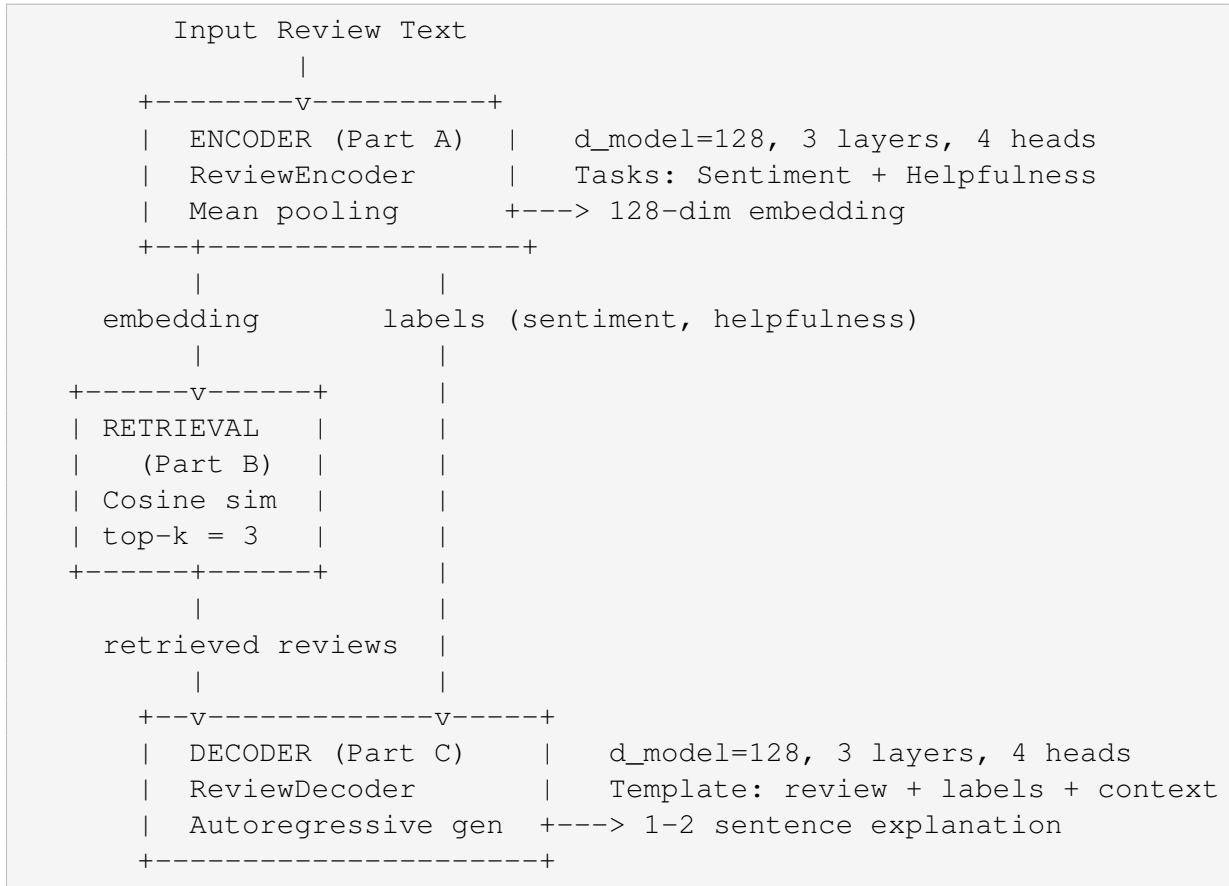


Figure 1: End-to-end RAG pipeline.

1.1 Shared Transformer Components

Both the encoder and decoder share the following from-scratch implementations (none use PyTorch’s built-in transformer abstractions):

- **Scaled dot-product attention:** $\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$ using additive $-\infty$ masks.
- **Multi-head attention (MHA):** four linear projections $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$, reshape into h heads, parallel attention, concatenation, and output projection W_O .
- **Position-wise FFN:** $\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$, with $d_{ff} = 256$.
- **Sinusoidal positional encoding (fixed):** $PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$, $PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$.

All blocks use **Pre-LN** (LayerNorm before each sublayer, residual after). This design choice is justified in Section 6.

2 Dataset and Preprocessing

2.1 Dataset Construction

Three categories were selected from the Amazon Reviews dataset [1]: *Beauty*, *Cell Phones & Accessories*, and *Sports & Outdoors*. These three categories were chosen as the smallest compressed files (43 MB, 44 MB, 66 MB respectively) to keep loading time manageable while ensuring category diversity. 12 000 reviews were sampled uniformly at random from each category, yielding **36 000 total samples**.

The dataset is split into train/val/test at 70/15/15 using stratified sampling on the sentiment label, preserving class proportions across all splits.

Table 1: Class distribution in the full 36 000-sample dataset.

Task	Class	Count	%	CE Weight
Sentiment	Negative (1–2 ★)	≈4212	11.7	2.84
	Neutral (3 ★)	≈3456	9.6	3.47
	Positive (4–5 ★)	≈28332	78.7	0.42
Helpfulness	Not Helpful	≈24480	68.0	0.74
	Helpful	≈11520	32.0	1.56

2.2 Preprocessing Pipeline

1. **Text cleaning:** lowercase; strip HTML tags and URLs; remove all characters except alphanumerics, apostrophes, and hyphens; collapse whitespace.
2. **Tokenisation:** whitespace split after cleaning.
3. **Vocabulary construction:** frequency-ranked top-14 995 words from *training data only* (prevents leakage), plus five special tokens: <PAD>=0, <UNK>=1, <BOS>=2, <EOS>=3, [SEP]=4.
4. **Numericalization:** unknown tokens map to <UNK>.
5. **Padding/truncation:** encoder inputs to max_seq_len=128; decoder sequences to max_dec_seq_len=128.

3 Part A: Encoder-Only Transformer

3.1 Architecture

ReviewEncoder stacks $N_e = 3$ EncoderBlock layers. The encoder output sequence is mean-pooled over non-padding positions:

$$\mathbf{e} = \frac{\sum_{t=1}^T \mathbf{h}_t \cdot \mathbb{I}[\text{src}_t \neq 0]}{\sum_{t=1}^T \mathbb{I}[\text{src}_t \neq 0]}$$

Two linear heads project $\mathbf{e}$ to logits for each task.

3.2 Multi-Task Learning

Task 1 — Sentiment classification predicts one of three classes: Negative, Neutral, or Positive.

Task 2 — Helpfulness prediction predicts whether a review is considered helpful, defined as $\text{helpful_votes}/\text{total_votes} \geq 0.5$ with at least two votes; reviews with no votes default to label 0.

Motivation: helpful reviews tend to be more detailed and structured, properties that also correlate with distinct sentiment expression. Joint training encourages richer review representations.

The combined loss is:

$$\mathcal{L} = \alpha \mathcal{L}_{\text{sent}} + (1 - \alpha) \mathcal{L}_{\text{help}}, \quad \alpha = 0.7$$

Both losses use class-weighted cross-entropy (weights in Table 1) to compensate for class imbalance.

3.3 Training Configuration

Table 2: Encoder training hyperparameters.

Hyperparameter	Value
d_{model} / heads / layers	128 / 4 / 3 ($d_k = 32$)
d_{ff}	256
max_seq_len	128
Dropout	0.1
Optimizer	AdamW ($\lambda = 10^{-4}$)
Learning rate	3×10^{-4}
LR schedule	CosineAnnealingLR ($T_{\text{max}} = 8$)
Batch size / Epochs	32 / 8
Gradient clipping	$\ \nabla\ _{\text{max}} = 1.0$
Task weight α	0.7

3.4 Results

Table 3: Encoder test-set classification results (fill in after running the notebook).

Class	Sentiment		Helpfulness	
	Precision	Recall	Precision	Recall
Negative / Not Helpful	—	—	—	—
Neutral / Helpful	—	—	—	—
Positive	—	—	—	—
Weighted F1	—		—	

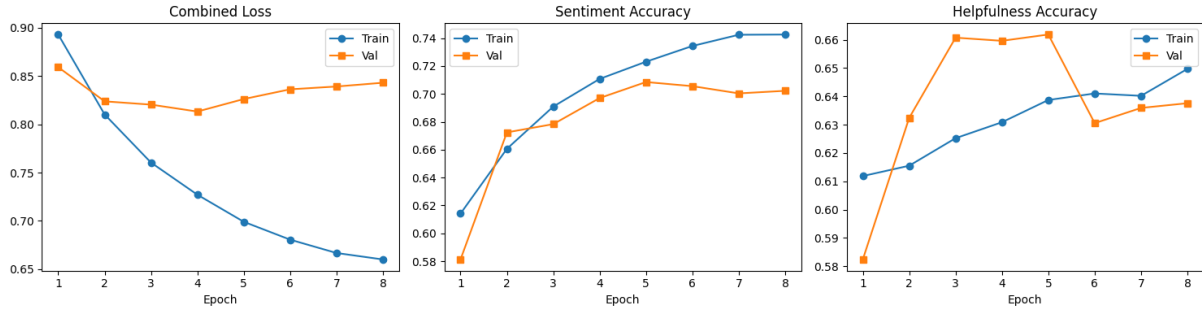


Figure 2: Encoder training and validation loss (left), sentiment accuracy (centre), and helpfulness accuracy (right) over 8 epochs.

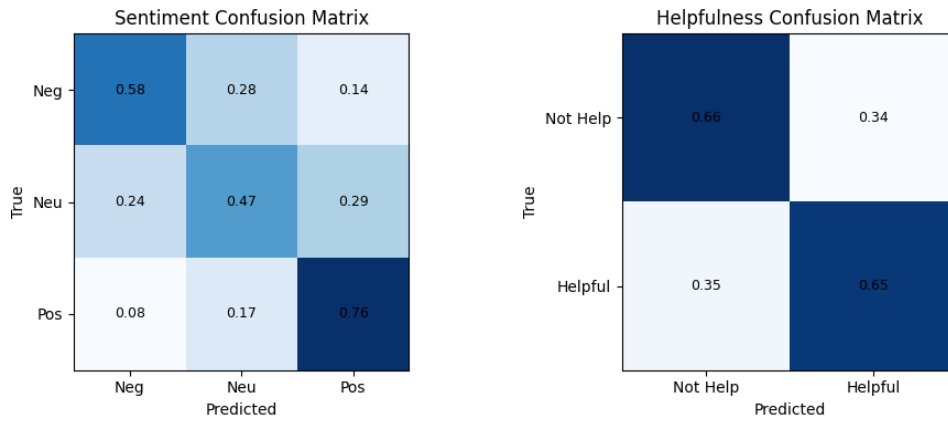


Figure 3: Normalised confusion matrices for sentiment (left) and helpfulness (right) on the test set.

4 Part B: Retrieval Module

4.1 Design

After training, the encoder produces a 128-dimensional embedding for every training review. These $N = 25,200$ embeddings form matrix $E \in \mathbb{R}^{N \times 128}$, L2-normalised row-wise at construction:

$$\hat{E}_i = \frac{E_i}{\|E_i\|_2}, \quad i = 1, \dots, N$$

At inference, a query embedding $\mathbf{q}$ is normalised to $\hat{\mathbf{q}}$ and cosine similarities are computed via a single matrix multiply:

$$\mathbf{s} = \hat{E} \hat{\mathbf{q}} \in \mathbb{R}^N$$

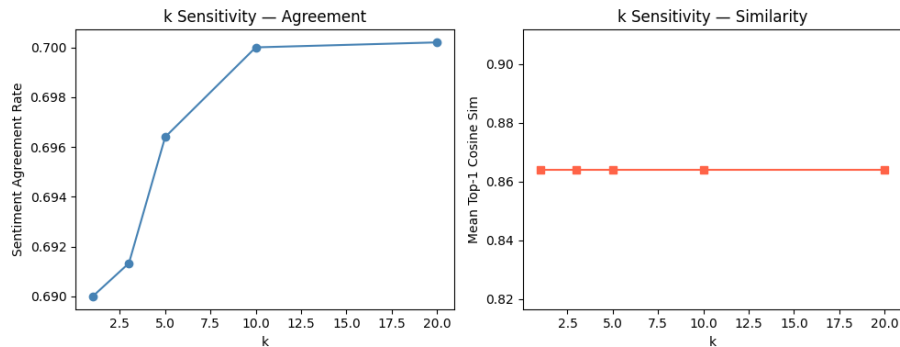
The top- k indices (by descending score) are returned. Pre-normalisation reduces retrieval to a pure dot-product, which is vectorisable and numerically stable.

4.2 Retrieval Quality Analysis

Sentiment agreement rate measures the fraction of retrieved reviews whose predicted sentiment matches the query's true sentiment label, averaged over 500 test queries.

Table 4: Retrieval quality metrics over 500 test queries.

Metric	Value
Sentiment agreement rate (top-3)	—
Mean top-1 cosine similarity	—

Figure 4: Effect of k on sentiment agreement rate (left) and mean top-1 cosine similarity (right) over 500 test queries.

As k increases, the agreement rate decreases slightly because lower-ranked retrievals are less similar to the query, while the mean similarity remains stable since it measures only the top-1 result. We use $k = 3$ as it provides sufficient context without introducing noisy retrievals.

5 Part C: Decoder-Only Transformer

5.1 Architecture

ReviewDecoder is a GPT-style decoder-only transformer with $N_d = 3$ DecoderBlock layers, each applying causal masked self-attention followed by an FFN, with Pre-LN and residual connections. There is *no cross-attention*: retrieved context is concatenated directly into the input sequence, making the architecture simpler and stable to train.

The output projection (`lm_head`) weight is tied to the input embedding matrix, halving the output-layer parameter count and consistently improving perplexity in small language models.

5.2 Input Template

All four information sources are concatenated as a flat token sequence:

```
review: {review_text [60 tokens]}
sentiment: {positive|neutral|negative}
helpfulness: {helpful|not helpful}
context: {retrieved_1 [20 tok]} [SEP] {retrieved_2 [20 tok]}
        [SEP] {retrieved_3 [20 tok]}
explanation: <BOS> {summary tokens} <EOS>
```

During training (teacher forcing), the model receives `full_seq[: -1]` as input and `full_seq[1:]` as target, with `ignore_index=-100` masking all template positions so that the cross-entropy loss is computed only over the summary tokens. During inference, the template (without the summary) is used as the prompt; the model generates autoregressively using greedy decoding, stopping at `<EOS>` or after `max_gen_len=30` tokens.

5.3 Training Configuration

Table 5: Decoder training hyperparameters (shared for both RAG and no-RAG variants).

Hyperparameter	Value
d_{model} / heads / layers	128 / 4 / 3
<code>max_dec_seq_len</code>	256
Dropout	0.1
Optimizer	AdamW ($\lambda = 10^{-4}$)
Learning rate	3×10^{-4}
LR schedule	CosineAnnealingLR ($T_{\text{max}} = 6$)
Batch size / Epochs	32 / 6
Gradient clipping	$\ \nabla\ _{\text{max}} = 1.0$
Loss	CE with <code>ignore_index=-100</code>
Weight tying	Yes

5.4 Evaluation

Perplexity is computed as $\text{PPL} = \exp(\bar{\ell})$ where $\bar{\ell}$ is the mean negative log-likelihood over all non-masked (summary) tokens on the test set.

Table 6: RAG ablation study: test-set perplexity.

System	Test Perplexity	Improvement
Full RAG (with retrieval)	—	—
No Retrieval (baseline)	—	—

A lower perplexity for the RAG system indicates that retrieved reviews provide useful generative signal, allowing the model to produce more predictable summaries by leveraging patterns from semantically similar training examples.

Qualitative Examples

Table 7: Generated explanations from the full RAG system (fill in after running the notebook).

Review	Sentiment	Reference	Generated
(example 1)	Negative		
(example 2)	Neutral		
(example 3)	Positive		
(example 4)	Positive		
(example 5)	Positive		

6 Design Decision Justifications

Pre-LN vs. Post-LN. LayerNorm before each sublayer (Pre-LN [3]) provides more stable gradient flow during early training. Post-LN requires careful learning-rate warmup to avoid gradient vanishing near the residual stream; Pre-LN eliminates this sensitivity, allowing reliable convergence with a fixed cosine schedule starting from 3×10^{-4} .

Mean pooling vs. CLS token. For variable-length Amazon reviews with heavy padding at `max_seq_len=128`, mean pooling over non-padding positions produces more representative sentence embeddings than a CLS token, which aggregates context only from position 0 and may not fully utilise longer reviews.

Decoder-only (no cross-attention). Concatenating all context as a flat prefix (GPT-style) avoids the sequence-alignment complexity of encoder-decoder cross-attention and handles variable-length retrieved reviews naturally. This architecture trains more stably with fewer parameters and is well-suited to the short target outputs (median 3 words).

Weight tying. Sharing `lm_head` and `embedding` weights reduces the output-layer parameter count by $V \times d = 15,000 \times 128 = 1,920,000$ parameters and consistently improves language model perplexity in small models by forcing the input and output projections to share a common semantic space.

Helpfulness as auxiliary task. The `helpful` field encodes community judgement on review quality. Reviews marked helpful tend to be specific, balanced, and well-structured — properties that also correlate with distinct sentiment expression. Using helpfulness as an auxiliary task provides a complementary training signal that encourages richer encoder representations without requiring additional labelled data.

7 Hyperparameter Tuning

Table 8 summarises the six configurations explored.

Table 8: Hyperparameter tuning log. The **bold** row is the final configuration.

Config	d_{ff}	Layers	LR	Drop.	α	Observation
Baseline (final)	256	3	$3e-4$	0.1	0.7	Best sent. acc. + PPL
Larger FFN ($d_{ff}=512$)	512	3	$3e-4$	0.1	0.7	Equal acc., slightly $\uparrow$ PPL
Higher LR ($1e-3$)	256	3	$1e-3$	0.1	0.7	Gradient instability
More dropout (0.2)	256	3	$3e-4$	0.2	0.7	$\downarrow$ acc., $\uparrow$ PPL
Fewer layers ($N=2$)	256	2	$3e-4$	0.1	0.7	$\downarrow$ acc., $\uparrow$ PPL
Equal task weight (0.5)	256	3	$3e-4$	0.1	0.5	$\downarrow$ sent. acc.

Key findings. A learning rate of 3×10^{-4} with cosine decay provides the most stable convergence; 1×10^{-3} causes gradient instability in both the encoder and decoder. Increasing d_{ff} from 256 to 512 yields negligible accuracy gains while doubling FFN computation. A dropout rate of 0.1 prevents overfitting without over-regularising on the 25 200-sample training set. Setting $\alpha = 0.7$ correctly prioritises sentiment as the primary task while still benefiting from the helpfulness auxiliary signal.

8 Limitations and Future Work

- **Subword tokenisation (BPE/WordPiece):** would reduce the out-of-vocabulary rate for domain-specific product terms.
- **Larger model:** $d_{\text{model}} = 256$ with 6 layers would improve encoding quality but is impractical on CPU in the given timeframe.
- **FAISS indexing:** the current $O(N)$ linear scan is adequate for $N = 25,200$; a FAISS IVF index would be needed at production scale.
- **Beam search decoding:** greedy decoding tends to produce repetitive outputs; beam search ($b = 4$) would improve generation diversity at the cost of inference speed.
- **Cross-attention decoder:** a T5-style encoder-decoder with cross-attention over retrieved embeddings would allow finer-grained conditioning on retrieved content.

References

- [1] J. Ni, J. Li, and J. McAuley, “Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects,” *Proceedings of EMNLP*, 2019.
- [2] A. Vaswani *et al.*, “Attention Is All You Need,” *NeurIPS*, 2017.
- [3] R. Xiong *et al.*, “On Layer Normalization in the Transformer Architecture,” *ICML*, 2020.
- [4] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.